

Tarea 2: Números Pseudoaleatorios y Métodos de Monte Carlo

Curso Libre de Configuración 2: Análisis de Datos

Herman Andrés Echeverría Rojas Facultad de Ingeniería, Universidad del Istmo de Guatemala

20 de septiembre de 2026

Resumen

Este documento acompaña al repositorio <https://github.com/HermanEcheverria/numeros-pseudoaleatorios-monte-carlo>. Todas las figuras y tablas que aparecen aquí son generadas por los scripts del repositorio a partir de las semillas registradas en `semillas.py`: no hay ningún número transcrito a mano. Cualquiera puede clonar el repositorio, ejecutar los scripts y obtener exactamente estos resultados.

Índice

I	Generadores de números pseudoaleatorios	3
1.	Fundamento común	3
1.1.	Qué es, formalmente, un generador pseudoaleatorio	3
1.2.	Toda sucesión pseudoaleatoria es periódica	4
1.3.	El generador congruencial lineal como caso de estudio	4
1.3.1.	Forma cerrada	4
1.3.2.	Período: el teorema de Hull–Dobell	5
1.3.3.	Los casos degenerados	5
1.4.	Estructura de red: por qué las k -tuplas caen en hiperplanos	6
1.5.	Normalización	7
2.	Los cinco generadores	7
2.1.	Generador congruencial lineal	7
2.2.	RANDU	8
2.3.	Método de los cuadrados medios	8
2.4.	Mersenne Twister (MT19937)	9
2.5.	Blum Blum Shub	10
2.6.	Resultados de los incisos (a) y (b)	12
2.7.	Robustez sobre múltiples semillas	13
3.	Prueba espectral	14
4.	Tabla comparativa	17
4.1.	Comparación con <code>random</code> y <code>secrets</code>	17
II	Métodos de Monte Carlo	17

5. Fundamento: el estimador y su error	18
5.1. La integral como valor esperado	18
5.2. El estimador es insesgado	18
5.3. La varianza y la tasa $O(N^{-1/2})$	19
5.4. Intervalo de confianza del 95 %	19
6. Integrales en una dimensión	20
6.1. Estudio de convergencia	20
6.2. Verificación de la cobertura	21
7. Alta dimensión y la maldición de la dimensionalidad	22
7.1. Comparación contra cuadratura determinística	22
7.2. Por qué el muestreo uniforme se vuelve ineficiente	23
8. Reducción de varianza	24
8.1. Variables antitéticas	24
8.2. Variables de control	24
8.3. Resultados	24
9. Cuando el generador arruina el resultado: RANDU	25
9.1. Dónde sí muerde	26

Parte I

Generadores de números pseudoaleatorios

1. Fundamento común

Los cinco algoritmos que pide el enunciado (LCG, cuadrados medios, Mersenne Twister, Blum Blum Shub y RANDU) parecen muy distintos entre sí, pero comparten el mismo esqueleto. Conviene aislarlo una sola vez: de ahí salen dos resultados —la periodicidad inevitable y la estructura de red— que después se aplican a cada generador en particular.

1.1. Qué es, formalmente, un generador pseudoaleatorio

Definición 1 (Generador pseudoaleatorio). *Un generador de números pseudoaleatorios es una cuádrupla $(\mathcal{S}, g, h, x_0)$ donde*

- $\mathcal{S}$ es un conjunto **finito** de estados,
- $g : \mathcal{S} \rightarrow \mathcal{S}$ es la función de transición,
- $h : \mathcal{S} \rightarrow [0, 1)$ es la función de salida, y
- $x_0 \in \mathcal{S}$ es la semilla.

El generador produce la sucesión de estados $x_{n+1} = g(x_n)$ y la sucesión de salidas $u_n = h(x_n)$.

Las tres piezas se reparten así en los algoritmos de esta tarea:

Algoritmo	Estado $\mathcal{S}$	Transición g	Salida h
LCG / RANDU	$\{0, \dots, m-1\}$	$x \mapsto (ax + c) \bmod m$	x/m
Cuadrados medios	enteros de $2k$ dígitos	dígitos centrales de x^2	$x/10^k$
Blum Blum Shub	$\mathbb{Z}_M^*$	$x \mapsto x^2 \bmod M$	bits bajos de x
Mersenne Twister	$(\mathbb{Z}_2^{32})^{624}$	torcedura lineal sobre $\mathbb{F}_2$	templado de una palabra

Tabla 1: Los cuatro esquemas bajo la misma estructura de la Definición 1.

Dos consecuencias se leen directamente de la definición, antes de programar nada:

Determinismo. Como g es una función y x_0 está fijo, toda la sucesión queda determinada por la semilla. Dos ejecuciones con la misma semilla producen *exactamente* la misma secuencia. Esto no es un defecto sino el requisito de reproducibilidad que exige el enunciado; por eso cada generador de este trabajo recibe su semilla como argumento explícito.

Ausencia de azar genuino. La sucesión (u_n) no es aleatoria en ningún sentido probabilístico: es una sucesión fija. Lo único que se puede pedir es que sea *indistinguible* de una muestra i.i.d. $\text{Unif}(0, 1)$ frente a una batería de pruebas estadísticas dada. Esta sección se ocupa de la parte determinista; las pruebas de hipótesis, de la otra.

1.2. Toda sucesión pseudoaleatoria es periódica

Teorema 1 (Periodicidad eventual). *Sea $(\mathcal{S}, g, h, x_0)$ un generador pseudoaleatorio con $|\mathcal{S}| = N < \infty$. Entonces existen enteros $\lambda \geq 0$ (la cola) y $\rho \geq 1$ (el período) tales que*

$$x_{n+\rho} = x_n \quad \text{para todo } n \geq \lambda,$$

y además $\lambda + \rho \leq N$.

Demostración. Considérense los $N + 1$ estados $x_0, x_1, \dots, x_N$. Todos pertenecen a $\mathcal{S}$, que tiene solo N elementos. Por el principio del palomar, dos de ellos coinciden: existen índices $0 \leq i < j \leq N$ con $x_i = x_j$.

Sea $\rho = j - i \geq 1$. Como x_{n+1} depende *únicamente* de x_n , aplicar g a ambos lados de $x_i = x_j$ da $x_{i+1} = x_{j+1}$, y por inducción $x_{i+k} = x_{j+k} = x_{i+k+\rho}$ para todo $k \geq 0$. Tomando $\lambda = i$ se obtiene $x_{n+\rho} = x_n$ para todo $n \geq \lambda$. Finalmente $\lambda + \rho = i + (j - i) = j \leq N$. $\square$

La demostración es corta pero dice tres cosas que importan en todo el trabajo:

1. **El período nunca supera el número de estados**, $\rho \leq N$. Aumentar el período obliga a agrandar el espacio de estados: es exactamente lo que hace el Mersenne Twister al guardar 624 palabras de 32 bits en lugar de un solo entero, alcanzando $\rho = 2^{19937} - 1$.
2. **Puede haber una cola** $\lambda > 0$: estados iniciales que nunca se vuelven a visitar. Esto ocurre cuando g no es inyectiva, que es justamente el caso de los cuadrados medios. Si g sí es biyectiva —como en un LCG con $\gcd(a, m) = 1$ — entonces $\lambda = 0$ y la sucesión es puramente periódica desde el inicio.
3. **La periodicidad es estructural, no un defecto de implementación.** Ningún algoritmo determinista sobre memoria finita puede evitarla. Lo único negociable es que ρ sea tan grande que no se alcance en la práctica.

1.3. El generador congruencial lineal como caso de estudio

El LCG es el más simple de los cinco y el único cuyo comportamiento se puede caracterizar por completo con álgebra elemental. Por eso se deriva aquí: RANDU resultará ser un caso particular suyo, y los defectos que se demuestren para la familia se heredan automáticamente.

La recurrencia es

$$x_{n+1} = (a x_n + c) \text{ mód } m, \quad n = 0, 1, 2, \dots \quad (1)$$

con *módulo* $m > 1$, *multiplicador* $0 < a < m$, *incremento* $0 \leq c < m$ y *semilla* $0 \leq x_0 < m$. Si $c = 0$ el generador se llama *multiplicativo* (o de Lehmer); si $c \neq 0$, *mixto*.

1.3.1. Forma cerrada

Lema 1 (Forma cerrada del LCG). *Para todo $n \geq 0$,*

$$x_n \equiv a^n x_0 + c \sum_{k=0}^{n-1} a^k \quad (\text{mód } m). \quad (2)$$

Demostración. Por inducción sobre n .

Base. Para $n = 0$ la suma es vacía (vale 0) y el enunciado se reduce a $x_0 \equiv a^0 x_0 = x_0$, que es cierto.

Paso inductivo. Supóngase (2) válida para n . Aplicando la recurrencia (1) y la hipótesis de inducción:

$$\begin{aligned} x_{n+1} &\equiv a x_n + c \equiv a \left(a^n x_0 + c \sum_{k=0}^{n-1} a^k \right) + c \\ &\equiv a^{n+1} x_0 + c \sum_{k=0}^{n-1} a^{k+1} + c \equiv a^{n+1} x_0 + c \sum_{k=1}^n a^k + c \\ &\equiv a^{n+1} x_0 + c \sum_{k=0}^n a^k \pmod{m}, \end{aligned}$$

donde el último paso absorbe el término $c = c a^0$ dentro de la suma. Esto es (2) para $n + 1$. $\square$

El Lema 1 tiene una lectura incómoda: x_n es una función *explícita y barata* de n . No hace falta iterar para saltar a cualquier punto de la sucesión, y —lo que es peor para cualquier uso criptográfico— basta observar unas pocas salidas consecutivas para plantear un sistema y recuperar (a, c, m) . Ningún LCG es seguro, y la razón es esta fórmula, no una debilidad de implementación.

1.3.2. Período: el teorema de Hull–Dobell

Por el Teorema 1 el período cumple $\rho \leq m$. La pregunta natural es cuándo se alcanza la igualdad.

Teorema 2 (Hull–Dobell, 1962 [1]). *Un LCG mixto (1) tiene período completo $\rho = m$ para toda semilla x_0 si y solo si se cumplen las tres condiciones:*

1. $\gcd(c, m) = 1$;
2. $a - 1$ es divisible por todo factor primo q de m ;
3. si $4 \mid m$, entonces $4 \mid (a - 1)$.

Corolario para módulos potencia de dos. Si $m = 2^k$ con $k \geq 2$, el único factor primo de m es 2 y $4 \mid m$, de modo que las tres condiciones colapsan en dos reglas verificables de un vistazo:

$$c \text{ impar} \quad \text{y} \quad a \equiv 1 \pmod{4}. \quad (3)$$

En efecto, $\gcd(c, 2^k) = 1 \iff c$ es impar; la condición (2) pide $2 \mid (a - 1)$ y la (3) pide $4 \mid (a - 1)$, de las cuales la segunda implica la primera. Este corolario explica de entrada por qué RANDU *no* alcanza período completo: al tener $c = 0$ falla la condición (1) desde el primer paso.

1.3.3. Los casos degenerados

Antes de programar conviene identificar qué elecciones de parámetros rompen el generador, porque son las que la implementación debe rechazar explícitamente.

$a = 0$. La recurrencia se vuelve $x_{n+1} = c \pmod{m}$ para todo n : la sucesión es la constante c/m desde el primer término. Período $\rho = 1$.

$c = 0$ y $x_0 = 0$. El cero es punto fijo de la recurrencia multiplicativa, pues $g(0) = (a \cdot 0) \pmod{m} = 0$. Toda la sucesión es idénticamente cero. Este caso es especialmente traicionero porque no lanza ningún error: produce diez mil ceros que pasan desapercibidos si uno no mira el histograma.

$a = 1$. La recurrencia se reduce a $x_n = (x_0 + nc)$ mód m , una progresión aritmética. Tiene período completo si $\gcd(c, m) = 1$, y sin embargo las salidas son una rampa perfectamente predecible. Es el contraejemplo que demuestra que **período completo no implica calidad**: Hull–Dobell es necesario pero muy lejos de suficiente.

1.4. Estructura de red: por qué las k -tuplas caen en hiperplanos

El defecto más serio de la familia LCG no lo detecta ninguna prueba unidimensional. Aparece al mirar las salidas de k en k .

Teorema 3 (Estructura de red [2]). *Sea un LCG con parámetros (a, c, m) y sea $k \geq 2$. Para todo vector de enteros $\mathbf{h} = (h_0, h_1, \dots, h_{k-1})$ que satisfaga*

$$h_0 + h_1 a + h_2 a^2 + \dots + h_{k-1} a^{k-1} \equiv 0 \pmod{m}, \quad (4)$$

los puntos $\mathbf{u}_n = (u_n, u_{n+1}, \dots, u_{n+k-1})$ del cubo $[0, 1)^k$ caen todos sobre una familia de hiperplanos paralelos, y el número de hiperplanos efectivamente ocupados es a lo sumo $\sum_i |h_i|$.

Demostración. Iterando el Lema 1 desde x_n en lugar de x_0 ,

$$x_{n+i} \equiv a^i x_n + c \sigma_i \pmod{m}, \quad \sigma_i := \sum_{j=0}^{i-1} a^j,$$

donde σ_i **no depende de n** . Entonces

$$\sum_{i=0}^{k-1} h_i x_{n+i} \equiv x_n \underbrace{\sum_{i=0}^{k-1} h_i a^i}_{\equiv 0 \text{ por (4)}} + c \underbrace{\sum_{i=0}^{k-1} h_i \sigma_i}_{=: K} \equiv cK \pmod{m}.$$

El lado izquierdo es congruente módulo m a una constante que no depende de n , así que existe un entero ℓ_n con $\sum_i h_i x_{n+i} = cK + \ell_n m$. Dividiendo entre m y usando $u_{n+i} = x_{n+i}/m$:

$$\sum_{i=0}^{k-1} h_i u_{n+i} = \frac{cK}{m} + \ell_n. \quad (5)$$

Para cada valor entero de ℓ_n , la ecuación (5) define un hiperplano en $[0, 1)^k$; al variar n lo único que cambia es ℓ_n , de modo que todos los puntos viven en esa familia de hiperplanos paralelos. Como $u_{n+i} \in [0, 1)$, el lado izquierdo de (5) está acotado en un intervalo de longitud $\sum_i |h_i|$, luego ℓ_n solo puede tomar a lo sumo $\sum_i |h_i|$ valores enteros distintos. $\square$

Marsaglia [2] demuestra además que *siempre* existe un $\mathbf{h}$ no trivial que cumple (4) con coeficientes pequeños, y que en consecuencia los puntos de cualquier LCG caen en no más de $(k! m)^{1/k}$ hiperplanos. Para $k = 3$ y $m = 2^{31}$ esa cota vale

$$(3! \cdot 2^{31})^{1/3} = (6 \cdot 2\,147\,483\,648)^{1/3} \approx 2344,$$

que para efectos prácticos es inofensivo: 2344 planos llenan el cubo de forma razonablemente densa. El escándalo de RANDU —que se documenta en su propia sección— es que se queda en **15**, dos órdenes de magnitud por debajo de lo que la cota permitía.

Lectura operativa. El Teorema 3 explica por qué el enunciado pide la *prueba espectral visual* y no solo pruebas de hipótesis. Las pruebas de uniformidad (χ^2 , Kolmogórov–Smirnov) miran una coordenada a la vez; las de independencia serial (autocorrelación, rachas) miran correlaciones lineales de a pares. Una relación exacta de tres términos como (4) es invisible para todas ellas y salta a la vista en el diagrama de dispersión tridimensional. Es la diferencia entre no encontrar evidencia y no haberla buscado donde estaba.

1.5. Normalización

Las salidas se obtienen dividiendo el estado entre el módulo, $u_n = x_n/m$. Se divide entre m y no entre $m - 1$ porque el estado recorre $\{0, 1, \dots, m - 1\}$: el valor máximo posible es $(m - 1)/m < 1$, que es exactamente el soporte $[0, 1)$ de la distribución $\text{Unif}(0, 1)$ estándar. Dividir entre $m - 1$ permitiría la salida $u = 1$, fuera del soporte, lo cual rompe cualquier transformación posterior que use $\log(1 - u)$ o la inversa de una función de distribución acumulada.

Conviene tener presente que la salida es un valor de una rejilla discreta de paso $1/m$, no un real del continuo. Ninguna implementación puede producir más de m valores distintos, y por eso una prueba de bondad de ajuste con más de m clases carecería de sentido.

2. Los cinco generadores

Cada generador se presenta con su fundamento matemático, su pseudocódigo, y los resultados de los incisos (a) y (b). Todas las semillas están registradas en `semillas.py` y aparecen en el pie de cada figura.

2.1. Generador congruencial lineal

El fundamento está derivado en la Sección 1.3: la recurrencia (1), la forma cerrada del Lema 1 y el Teorema 2 de Hull–Dobell.

Algoritmo 1 Generador congruencial lineal

Entrada: n , semilla x_0 , multiplicador a , incremento c , módulo m

Salida: n valores en $[0, 1)$

```

1: si  $a = 0$  o  $a = 1$  o ( $c = 0$  y  $x_0 = 0$ ) entonces
2:   error: caso degenerado
3: fin si
4:  $x \leftarrow x_0$ 
5: para  $i = 1$  hasta  $n$  hacer
6:    $x \leftarrow (a x + c) \bmod m$ 
7:    $u_i \leftarrow x/m$ 
8: fin para
9: devolver  $(u_1, \dots, u_n)$ 

```

Se usan los parámetros de *Numerical Recipes*: $a = 1\,664\,525$, $c = 1\,013\,904\,223$, $m = 2^{32}$. El corolario (3) se verifica de un vistazo: c es impar y $a - 1 = 1\,664\,524 = 4 \cdot 416\,131$, así que $a \equiv 1 \pmod{4}$. El período es completo, 2^{32} .

2.2. RANDU

RANDU es el LCG con $a = 65\,539$, $c = 0$, $m = 2^{31}$. Al ser $c = 0$ falla la primera condición de Hull–Dobell, y su período es el orden multiplicativo de a módulo 2^{31} , que resulta ser 2^{29} : desperdicia el 75 % de los estados disponibles.

Su defecto nace de que $65\,539 = 2^{16} + 3$. Elevando al cuadrado,

$$a^2 = (2^{16} + 3)^2 = 2^{32} + 6 \cdot 2^{16} + 9 = 6(2^{16} + 3) - 9 + 2^{32} = 6a - 9 + 2^{32},$$

y como $2^{32} \equiv 0 \pmod{2^{31}}$, queda $a^2 \equiv 6a - 9$. Multiplicando por x_n se obtiene la identidad exacta

$$x_{n+2} = 6x_{n+1} - 9x_n \pmod{2^{31}}. \quad (6)$$

Esta es la condición (4) del Teorema 3 con $\mathbf{h} = (9, -6, 1)$, y de ella salen los planos de la Sección 3.

Verificación numérica. La identidad (6) se comprobó sobre los estados enteros (no normalizados, para que el redondeo de punto flotante no la contamine) de una secuencia de 10 000 valores: se cumple en las **9 998** ternas, sin una sola excepción.

Sensibilidad a la semilla. Como $c = 0$, se tiene $x_n = a^n x_0 \pmod{m}$. Si la semilla es $x_0 = 2^s \cdot (\text{impar})$, el factor 2^s sale de la recurrencia y el generador se comporta como si el módulo fuera 2^{31-s} : el período se parte a la mitad con cada factor 2.

Semilla	Período	Salidas distintas de 64
1	$536\,870\,912 = 2^{29}$	64
2	268 435 456	64
2^{16}	8 192	64
2^{28}	2	2
2^{30}	1	1

Tabla 2: Degradación del período de RANDU según los factores de 2 de la semilla.

El caso extremo es contundente: con semilla $2^{30} = 1\,073\,741\,824$, **RANDU devuelve 0.5 para siempre**. La razón cabe en una línea: como 65 539 es impar, $65\,539 \cdot 2^{30} \pmod{2^{31}} = 2^{30}$, así que la semilla es un punto fijo. Un generador anunciado con período 2^{29} entrega una constante, y ninguna prueba de uniformidad avisa, porque el histograma es una sola barra. Por eso la semilla usada en este trabajo es impar y suficientemente grande para que $a x_0$ supere el módulo desde el primer paso.

2.3. Método de los cuadrados medios

Propuesto por von Neumann en 1949. El estado es un entero de k dígitos (con k par); se eleva al cuadrado, se rellena con ceros a la izquierda hasta $2k$ posiciones y se extraen los k dígitos centrales.

Algoritmo 2 Cuadrados medios (von Neumann)

Entrada: n , semilla x_0 de k dígitos, k par

Salida: n valores en $[0, 1)$

```
1:  $x \leftarrow x_0$ 
2: para  $i = 1$  hasta  $n$  hacer
3:    $s \leftarrow$  cadena de  $x^2$  rellena con ceros a la izquierda hasta  $2k$  dígitos
4:    $x \leftarrow$  entero formado por los dígitos  $s_{k/2+1}, \dots, s_{k/2+k}$ 
5:    $u_i \leftarrow x/10^k$ 
6: fin para
7: devolver  $(u_1, \dots, u_n)$ 
```

No hay teoría de período. A diferencia del LCG, este método carece de un resultado tipo Hull–Dobell: el período depende de la semilla de un modo que solo se conoce midiéndolo. La implementación se validó primero contra dos secuencias publicadas: la de von Neumann con $k = 2$ y semilla 43 ($43 \rightarrow 84 \rightarrow 05 \rightarrow 02 \rightarrow 00$, la «trampa del cero» en cuatro pasos) y el ciclo corto de $k = 4$ ($6100 \rightarrow 2100 \rightarrow 4100 \rightarrow 8100 \rightarrow 6100$). Ambas se reproducen exactamente.

Censo exhaustivo. Con $k = 4$ hay solo 10 000 semillas posibles, así que el período no se estima: se recorre el espacio completo.

Período	Semillas	Porcentaje
1	2,263	22.6%
4	7,737	77.4%

Tabla 3: Período de las 10 000 semillas de 4 dígitos. Barrido exhaustivo.

El período máximo sobre todas las semillas posibles es 4. No existe una sola semilla de 4 dígitos que produzca algo mejor. Las 2 263 de período 1 caen en cinco puntos fijos distintos: el 0 (que absorbe 1 968 semillas y es el «mecanismo cero» clásico), 2500 (130), 100 (104), 7600 (60) y 3792, que es un caso curioso: $3792^2 = 14\,379\,264$, cuyos cuatro dígitos centrales son 3792, y ninguna otra semilla llega a él.

Para producir 10 000 valores sin repetición hace falta subir a 12 dígitos. Con la semilla registrada, la órbita tiene una **cola de 326 565 estados** antes de entrar en un **ciclo de 2 500**. Esa cola es la $\lambda > 0$ del Teorema 1: existe porque la función de transición no es inyectiva, cosa imposible en un LCG con $\gcd(a, m) = 1$.

2.4. Mersenne Twister (MT19937)

Recurrencia lineal sobre $\mathbb{F}_2$ con 624 palabras de 32 bits de estado y período $2^{19937} - 1$, un primo de Mersenne. Tiene tres etapas.

Algoritmo 3 MT19937

Entrada: semilla s de 32 bits

Inicialización

```
1:  $mt[0] \leftarrow s$ 
2: para  $i = 1$  hasta 623 hacer
3:    $mt[i] \leftarrow (1\,812\,433\,253 \cdot (mt[i-1] \oplus (mt[i-1] \gg 30)) + i) \bmod 2^{32}$ 
4: fin para
   Torcedura (cuando se agotan las 624 palabras)
5: para  $i = 0$  hasta 623 hacer
6:    $x \leftarrow (mt[i] \wedge 0x80000000) \vee (mt[(i+1) \bmod 624] \wedge 0x7FFFFFFF)$ 
7:    $x_A \leftarrow x \gg 1$ ; si  $x$  es impar entonces  $x_A \leftarrow x_A \oplus 0x9908B0DF$ 
8:    $mt[i] \leftarrow mt[(i+397) \bmod 624] \oplus x_A$ 
9: fin para
   Templado (al emitir cada palabra)
10:  $y \leftarrow mt[\text{índice}]$ 
11:  $y \leftarrow y \oplus (y \gg 11)$ 
12:  $y \leftarrow y \oplus ((y \ll 7) \wedge 0x9D2C5680)$ 
13:  $y \leftarrow y \oplus ((y \ll 15) \wedge 0xEFC60000)$ 
14:  $y \leftarrow y \oplus (y \gg 18)$ 
15: devolver  $y/2^{32}$ 
```

Verificación. La especificación publica la salida para la semilla de referencia 5489. La implementación la reproduce exactamente:

i	Implementación	Especificación	Coincide
1	3499211612	3499211612	Sí
2	581869302	581869302	Sí
3	3890346734	3890346734	Sí
4	3586334585	3586334585	Sí
5	545404204	545404204	Sí

Tabla 4: Primeras palabras de 32 bits con semilla 5489, contra el vector de referencia de la especificación (OEIS A221557).

Las diez coinciden, y además coinciden con las palabras crudas de `numpy.random.RandomState(5489)`. Dos fuentes independientes.

Equidistribución y seguridad. MT19937 está equidistribuido en 623 dimensiones: sus k -tuplas llenan el hipercubo uniformemente hasta $k = 623$. Ese es el contraste exacto con RANDU, que falla ya en $k = 3$. Sin embargo **no es criptográficamente seguro**: el templado es una transformación invertible, así que observando 624 salidas consecutivas se reconstruye el estado interno y se predice todo el resto.

2.5. Blum Blum Shub

Recurrencia $x_{n+1} = x_n^2 \bmod M$ con $M = pq$ y p, q primos de Blum ($\equiv 3 \bmod 4$).

Algoritmo 4 Blum Blum Shub

Entrada: n , semilla s coprima con M , primos de Blum p, q

- 1: $M \leftarrow pq$; $k \leftarrow \lfloor \log_2 \log_2 M \rfloor$
 - 2: $x \leftarrow s^2 \bmod M$ $\triangleright$ garantiza que x sea residuo cuadrático
 - 3: **mientras** falten salidas **hacer**
 - 4: $x \leftarrow x^2 \bmod M$
 - 5: acumular los k bits menos significativos de x
 - 6: **si** hay 32 bits acumulados **entonces** emitir su valor dividido entre 2^{32}
 - 7: **fin mientras**
-

Condición	Se cumple
$p = 2147483783$ es primo	Sí
$q = 2148484187$ es primo	Sí
$p = 3 \pmod{4}$: $p \bmod 4 = 3$	Sí
$q = 3 \pmod{4}$: $q \bmod 4 = 3$	Sí
p es seguro: $(p-1)/2 = 1073741891$ primo	Sí
q es seguro: $(q-1)/2 = 1074242093$ primo	Sí
$\gcd((p-3)/2, (q-3)/2) = 2$ es pequeño	Sí

Tabla 5: Condiciones verificadas sobre los primos elegidos.

El período. Como $x_n = x_0^{2^n} \bmod M$, la sucesión se repite cuando 2^n recupera su valor módulo el orden de x_0 . El cálculo tiene dos trampas. Primero, el orden *no* se toma módulo $\lambda(M) = 2p'q'$, que es par y donde 2 ni siquiera es invertible; como x_0 es residuo cuadrático, su orden divide $\lambda(M)/2 = p'q'$, que es impar. Segundo, el orden de 2 módulo $p'q'$ divide el exponente del grupo de unidades, $\text{lcm}(p' - 1, q' - 1)$, y no $p'q'$. Con los primos elegidos el período resulta $2\,353\,997\,416\,697\,212 \approx 2^{51}$.

La fórmula se verificó contra el período observado iterando, con primos seguros pequeños: $(p, q) = (11, 23)$ da 20 predicho y 20 observado; $(23, 47)$ da 110 y 110; $(47, 59)$ da 308 y 308.

Seguridad y costo. Es el único de los cinco criptográficamente seguro: distinguir su salida de una secuencia aleatoria es tan difícil como resolver el problema de la residuosidad cuadrática módulo M , que se apoya en la dificultad de factorizar M . La garantía exige emitir a lo sumo $\lfloor \log_2 \log_2 M \rfloor = 5$ bits por estado; emitir el estado completo la destruiría. De ahí que necesite varias exponenciaciones modulares por número, contra una multiplicación del LCG, y que sea diez veces más lento.

2.6. Resultados de los incisos (a) y (b)

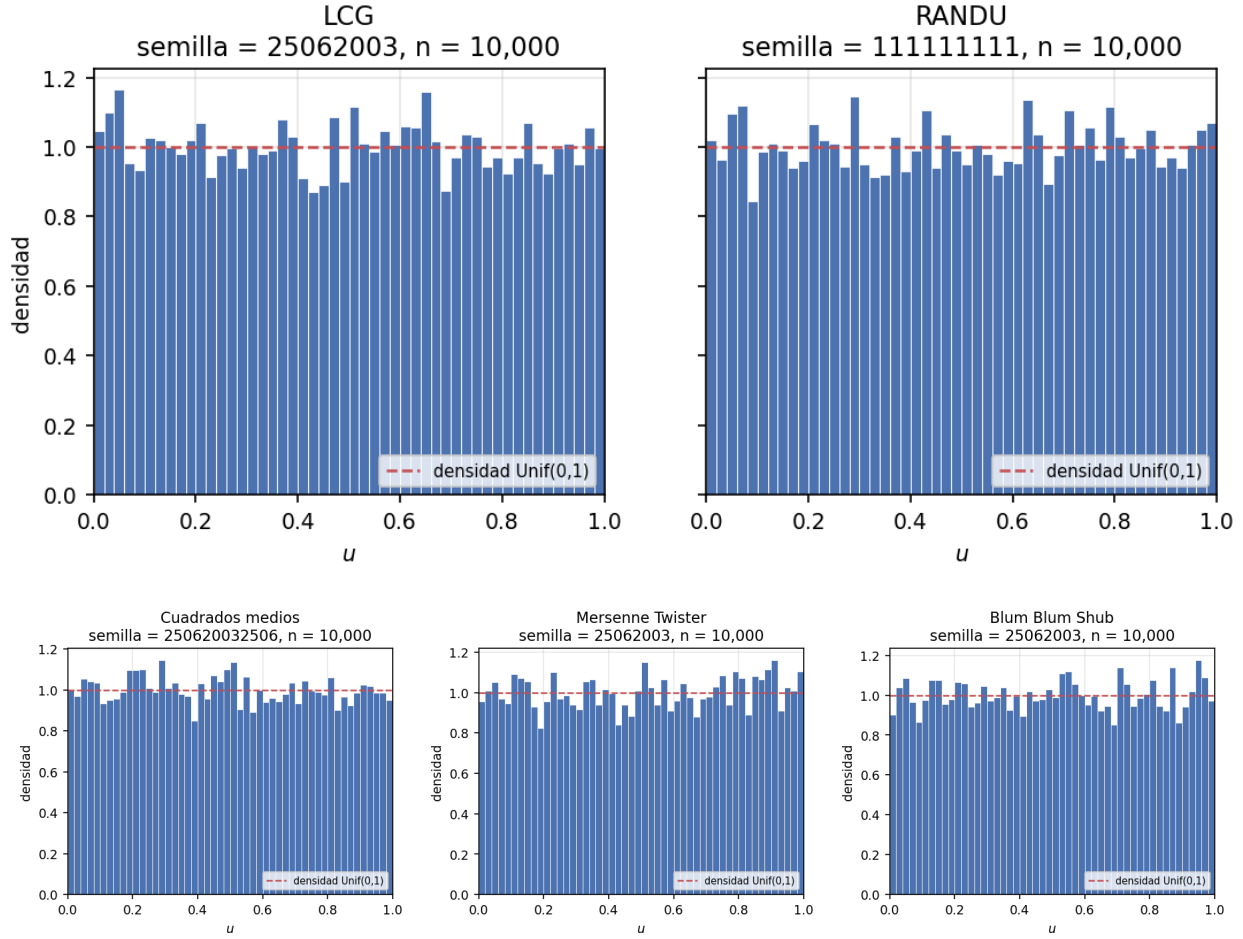


Figura 1: Histogramas normalizados de los cinco generadores, $n = 10000$, con la densidad teórica $\text{Unif}(0, 1)$ superpuesta.

Generador	Prueba	Estadístico	p -valor	Decisión ($\alpha = 0.05$)
LCG	χ^2	44.2400	0.6662	No se rechaza
LCG	K-S	0.0075	0.6306	No se rechaza
LCG	Autocorr.	0.0102	0.3087	No se rechaza
LCG	Rachas	4965.0000	0.4715	No se rechaza
RANDU	χ^2	42.6600	0.7265	No se rechaza
RANDU	K-S	0.0072	0.6732	No se rechaza
RANDU	Autocorr.	-0.0210	0.0353	Se rechaza
RANDU	Rachas	5030.0000	0.5619	No se rechaza

Tabla 6: Pruebas de hipótesis para LCG y RANDU.

Prueba	Estadístico	p -valor	Decisión ($\alpha = 0.05$)	Prueba	Estadístico	p -valor	Decisión ($\alpha = 0.05$)
χ^2	38.9000	0.8489	No se rechaza	χ^2	61.1500	0.1142	No se rechaza
K-S	0.0109	0.1823	No se rechaza	K-S	0.0125	0.0853	No se rechaza
Autocorr.	0.0076	0.4464	No se rechaza	Autocorr.	0.0005	0.9599	No se rechaza
Rachas	4964.0000	0.4593	No se rechaza	Rachas	5000.0000	0.9840	No se rechaza
Cuadrados medios				Mersenne Twister			
Prueba	Estadístico	p -valor	Decisión ($\alpha = 0.05$)				
χ^2	55.4700	0.2440	No se rechaza				
K-S	0.0062	0.8305	No se rechaza				
Autocorr.	0.0129	0.1959	No se rechaza				
Rachas	4979.0000	0.6599	No se rechaza				
Blum Blum Shub							

Tabla 7: Pruebas de hipótesis para los tres generadores restantes.

Interpretación. Los cinco generadores pasan las cuatro pruebas. Pasa el criptográficamente seguro y pasa el que tiene cinco puntos fijos y un período máximo de 4 con cuatro dígitos. Pasa RANDU, que lleva dentro la identidad exacta (6).

Eso no es un defecto de las pruebas sino su alcance: χ^2 y Kolmogórov–Smirnov examinan una coordenada a la vez, y la autocorrelación de retardo 1 y las rachas examinan relaciones entre pares consecutivos. Una relación de tres términos es invisible para las cuatro. **Una batería de pruebas certifica únicamente aquello que las pruebas miran.**

2.7. Robustez sobre múltiples semillas

Una sola corrida puede engañar en ambas direcciones. Con la semilla registrada, RANDU *rechaza* la prueba de autocorrelación con $p = 0.0353$, lo que invitaría a concluir que esa prueba detectó su defecto. Para comprobarlo se aplicó el esquema de dos niveles de NIST SP 800-22: la batería se repite sobre $K = 100$ semillas y se analiza la distribución de los p -valores.

Las 100 semillas no pueden elegirse consecutivas. En un LCG de período completo todas las semillas viven en el mismo ciclo, y dos semillas cercanas dan secuencias que se solapan casi por completo. Se obtienen saltando exactamente 10 000 posiciones mediante la forma cerrada del Lema 1, lo que las deja disjuntas por construcción; el salto se verificó contra iteración directa para los tres juegos de parámetros.

Generador	Prueba	Rechazos	Banda NIST	Dentro	χ^2 unif.	p
LCG	χ^2	3.0%	[0.0%, 11.5%]	Sí	11.40	0.2493
LCG	K-S	4.0%	[0.0%, 11.5%]	Sí	16.00	0.0669
LCG	Autocorr.	1.0%	[0.0%, 11.5%]	Sí	12.40	0.1917
LCG	Rachas	4.0%	[0.0%, 11.5%]	Sí	5.20	0.8165
RANDU	χ^2	3.0%	[0.0%, 11.5%]	Sí	6.40	0.6993
RANDU	K-S	4.0%	[0.0%, 11.5%]	Sí	10.60	0.3041
RANDU	Autocorr.	3.0%	[0.0%, 11.5%]	Sí	11.00	0.2757
RANDU	Rachas	6.0%	[0.0%, 11.5%]	Sí	13.40	0.1453

Tabla 8: Tasa de rechazo sobre 100 semillas disjuntas y uniformidad de los p -valores. La banda de NIST es $\alpha \pm 3\sqrt{\alpha(1-\alpha)/K}$.

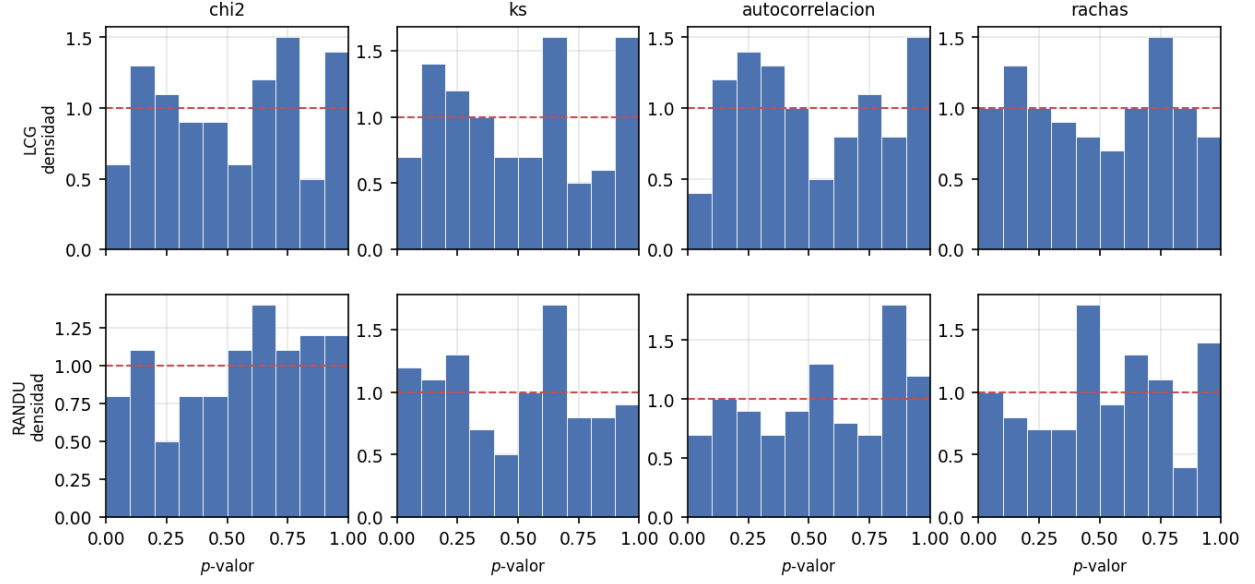


Figura 2: Distribución de los p -valores sobre 100 semillas. Bajo H_0 deben ser $\text{Unif}(0,1)$, es decir planos en 1.

La tasa de rechazo de RANDU en la prueba de autocorrelación es del 3%, dentro de la banda: aquel rechazo aislado fue un **error de tipo I**, no una detección. Las ocho tasas caen dentro de lo esperado y los ocho histogramas son compatibles con la uniformidad. Con 800 p -valores en lugar de 8, la conclusión se sostiene mucho mejor: estas cuatro pruebas no encuentran nada malo en RANDU.

3. Prueba espectral

El Teorema 3 predice que las ternas de un LCG caen en planos. Se grafican las ternas consecutivas (u_i, u_{i+1}, u_{i+2}) en el cubo $[0,1)^3$.

El ángulo de vista está derivado, no ajustado. La cámara apunta en la dirección $\mathbf{d} = (\cos e \cos a, \cos e \sin a, \sin e)$, donde e es la elevación y a el azimut. Los planos se ven de canto cuando se mira *paralelo* a ellos, es decir cuando $\mathbf{d}$ es perpendicular a la normal: $\mathbf{d} \cdot \mathbf{h} = 0$ con $\mathbf{h} = (9, -6, 1)$. Resolviendo numéricamente se obtiene $e = 47^\circ$, $a = 62^\circ$, con residuo 5×10^{-5} .

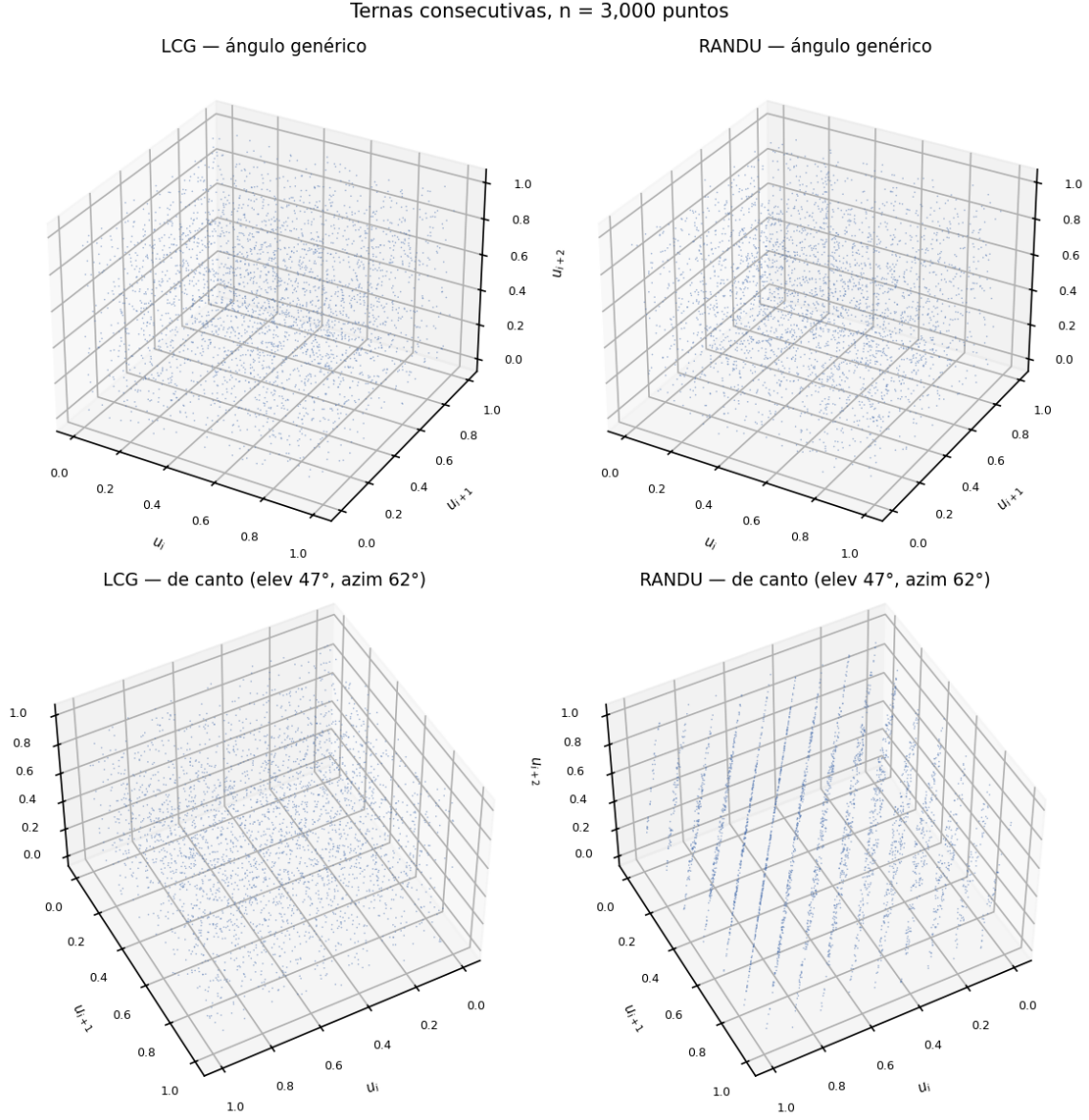


Figura 3: Ternas consecutivas, 3 000 puntos. Arriba, el ángulo por defecto: ambos generadores parecen ruido. Abajo, el ángulo derivado: RANDU colapsa en líneas.

La fila superior es tan importante como la inferior. Desde un ángulo genérico RANDU y el LCG son indistinguibles a simple vista: el defecto solo aparece si se mira desde la dirección correcta.

La demostración cuantitativa. La gráfica convence; el conteo demuestra. Para cada terna se calcula $k_i = 9u_i - 6u_{i+1} + u_{i+2}$, que por (6) debe ser entero en RANDU.

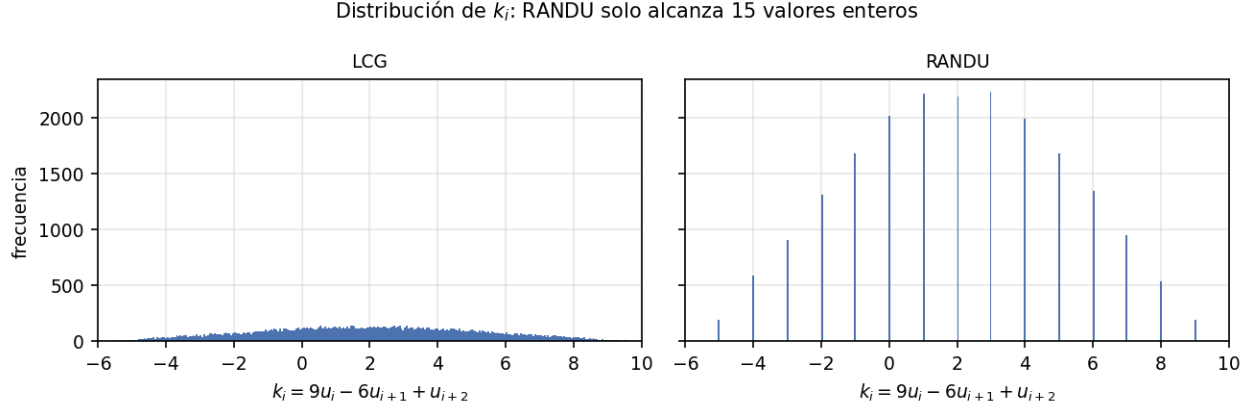


Figura 4: Distribución de k_i sobre 20 000 ternas.

Generador	Planos ocupados	Rango de k_i	Desv. máx. al entero
LCG	0	$[-5.904, 9.790]$	5.0e-01
RANDU	15	$[-5.000, 9.000]$	0.0e+00

Tabla 9: Conteo de planos ocupados.

RANDU toma exactamente **15** valores, $k \in \{-5, \dots, 9\}$, con desviación 0.00 respecto al entero más cercano: la relación es algebraica, no aproximada. El LCG de Numerical Recipes toma 0 valores enteros y su k_i se distribuye de forma continua.

El conteo coincide con la predicción del Teorema 3: como cada $u \in [0, 1)$, la combinación $9u_i - 6u_{i+1} + u_{i+2}$ está acotada entre -6 y 10 , así que solo caben los 15 enteros de -5 a 9 . Se predijeron 15 y se observaron 15.

Relación con el período y la calidad. RANDU tiene período $2^{29} \approx 5.4 \times 10^8$: más de quinientos millones de valores antes de repetirse. Y sus ternas ocupan 15 planos. Son propiedades **independientes**: el período mide cuánto tarda la secuencia en repetirse y la prueba espectral mide qué tan bien llena el espacio. Un generador puede tardar una eternidad en repetirse y distribuir pésimo en tres dimensiones.

Peor aún, la cota de Marsaglia permitía hasta $(3! \cdot 2^{31})^{1/3} \approx 2344$ planos, que habría sido inofensivo. RANDU se queda en 15: unas 150 veces por debajo de su propio peor caso permitido.

4. Tabla comparativa

Algoritmo	s / 10 ⁶	Rel.	Período	Seguridad	Casos de uso
LCG (Num. Recipes)	0.15	1.0×	2 ³²	Nula: la forma cerrada permite recuperar (a, c, m) con pocas salidas.	Simulación sencilla, videojuegos, sistemas embebidos, rand() de C.
RANDU	0.15	1.0×	2 ²⁹	Nula, y además las ternas caen en 15 planos.	Ninguno. Solo interés histórico y didáctico.
Cuadrados medios	0.29	1.9×	cola 326 565 + ciclo 2 500	Nula. Degenera: cinco puntos fijos con 4 dígitos.	Ninguno. Obsoleto desde los años 50.
Mersenne Twister	0.66	4.4×	2 ¹⁹⁹³⁷ - 1	Nula: con 624 salidas se reconstruye el estado.	Estándar de simulación. Motor de random de Python.
Blum Blum Shub	1.47	9.8×	≈ 2 ⁵¹	Criptográfica: equivale a la residuosidad cuadrática módulo M .	Criptografía, donde la lentitud es aceptable.

Tabla 10: Comparación de los cinco generadores. Las velocidades están cronometradas en esta máquina (mejor de tres corridas); los períodos son derivados donde hay teoría y medidos donde no la hay.

El orden de velocidad reproduce el costo algebraico de cada recurrencia. El LCG y RANDU son idénticos en costo, como debe ser: RANDU *es* un LCG. Los cuadrados medios pagan la conversión a cadena de dígitos. MT19937 paga la torcedura de 624 palabras. Y BBS es diez veces más lento que el LCG porque necesita varias exponenciaciones modulares para producir un solo número, ya que solo puede emitir 5 bits por estado.

4.1. Comparación con **random** y **secrets**

Para el inciso (d) se eligió MT19937 por una razón concreta: **random** de Python *es* MT19937. Al tratarse del mismo algoritmo, cualquier diferencia mide la implementación y no la calidad del generador.

Fuente	s / 10 ⁶	Reproducibile	χ^2	K-S	Autocorr.	Rachas
MT19937 (propio)	0.66	Sí	0.1142	0.0853	0.9599	0.9840
random (MT19937 de C)	0.06	Sí	0.3267	0.8335	0.8447	0.8729
secrets (SystemRandom)	0.26	No	0.4198	0.3069	0.6703	0.5755

Tabla 11: Comparación del MT19937 propio contra las librerías estándar.

La implementación propia es unas doce veces más lenta que **random**, y eso no dice nada sobre el algoritmo: es Python interpretado contra C compilado. El detalle revelador es que **secrets** resulta *más rápido* que el MT19937 propio pese a ser criptográficamente seguro. Tampoco es una ventaja algorítmica: **secrets** delega en el generador del sistema operativo, que también está en C. Lo caro no es la criptografía, es el bucle de Python.

La diferencia que sí es de diseño está en la reproducibilidad. **secrets** no acepta semilla, y no por omisión sino a propósito: un generador criptográfico predecible no sirve de nada. Eso lo vuelve inutilizable para simulación reproducible, que es justamente lo que este trabajo exige.

Parte II

Métodos de Monte Carlo

5. Fundamento: el estimador y su error

Esta sección deriva los tres resultados que sostienen todo lo que sigue: que el estimador de Monte Carlo es insesgado, que su error decae como $O(N^{-1/2})$ sin importar la dimensión, y cómo se construye su intervalo de confianza.

5.1. La integral como valor esperado

Sea f integrable en $[a, b]$ y sea $I = \int_a^b f(x) dx$. Si $U \sim \text{Unif}(a, b)$, su densidad es $p(x) = 1/(b-a)$ sobre $[a, b]$ y cero fuera. Entonces, por definición de valor esperado,

$$\mathbb{E}[f(U)] = \int_a^b f(x) p(x) dx = \frac{1}{b-a} \int_a^b f(x) dx = \frac{I}{b-a}, \quad (7)$$

de donde

$$\boxed{I = (b-a) \mathbb{E}[f(U)]} \quad (8)$$

El paso es puramente algebraico, pero cambia la naturaleza del problema: calcular una integral pasa a ser *estimar una media*. Y una media se estima promediando muestras.

5.2. El estimador es insesgado

Sean $U_1, \dots, U_N$ independientes e idénticamente distribuidas $\text{Unif}(a, b)$, y defínase

$$\hat{I}_N = \frac{b-a}{N} \sum_{i=1}^N f(U_i). \quad (9)$$

Teorema 4 (Inssegadez). $\mathbb{E}[\hat{I}_N] = I$ para todo $N \geq 1$.

Demostración. Por linealidad del valor esperado, que no requiere independencia:

$$\mathbb{E}[\hat{I}_N] = \frac{b-a}{N} \mathbb{E} \left[\sum_{i=1}^N f(U_i) \right] = \frac{b-a}{N} \sum_{i=1}^N \mathbb{E}[f(U_i)].$$

Como todas las U_i tienen la misma distribución, cada sumando vale lo mismo, $\mathbb{E}[f(U_i)] = \mathbb{E}[f(U)] = I/(b-a)$ por (7). Hay N sumandos iguales, así que

$$\mathbb{E}[\hat{I}_N] = \frac{b-a}{N} \cdot N \cdot \frac{I}{b-a} = I. \quad \square$$

Conviene subrayar qué *no* dice el teorema. Inssegado significa que el estimador acierta *en promedio sobre infinitas repeticiones*, no que una corrida particular esté cerca. Cuán cerca está una corrida es justamente lo que mide la varianza.

5.3. La varianza y la tasa $O(N^{-1/2})$

Teorema 5 (Varianza del estimador). Si $\sigma_f^2 := \text{Var}(f(U)) < \infty$, entonces

$$\text{Var}(\hat{I}_N) = \frac{(b-a)^2 \sigma_f^2}{N}, \quad \text{y por lo tanto} \quad \text{EE}(\hat{I}_N) = \frac{(b-a) \sigma_f}{\sqrt{N}}. \quad (10)$$

Demostración. Aquí sí hace falta la independencia. Para variables independientes la varianza de la suma es la suma de las varianzas, y $\text{Var}(cX) = c^2 \text{Var}(X)$:

$$\text{Var}(\hat{I}_N) = \left(\frac{b-a}{N} \right)^2 \text{Var} \left(\sum_{i=1}^N f(U_i) \right) = \frac{(b-a)^2}{N^2} \sum_{i=1}^N \text{Var}(f(U_i)) = \frac{(b-a)^2}{N^2} \cdot N \sigma_f^2,$$

que se simplifica a (10). El error estándar es la raíz. $\square$

El error estándar decae como $N^{-1/2}$. Es una tasa **lenta**: para ganar un dígito decimal de precisión hay que multiplicar N por cien. Pero tiene una propiedad que la vuelve valiosa.

Por qué la tasa no depende de la dimensión. Obsérvese qué entra en la fórmula (10): solamente N y σ_f^2 . La dimensión *no aparece*. La razón es que la derivación nunca usó que U fuera un número real: solo usó que las $f(U_i)$ fueran independientes, idénticamente distribuidas y de varianza finita. Si U es un punto en $\mathbb{R}^d$, cada $f(U_i)$ sigue siendo un *escalar*, el promedio sigue siendo un promedio de N números, y el mismo cálculo se aplica palabra por palabra.

El contraste con los métodos determinísticos es lo que da valor a esto. Una regla de cuadratura sobre una rejilla de m puntos por eje necesita m^d evaluaciones, y su error típico es $O(m^{-k}) = O(N^{-k/d})$ para un método de orden k : la tasa *se degrada* al crecer d . Monte Carlo mantiene $N^{-1/2}$ para todo d . Existe una dimensión a partir de la cual Monte Carlo gana, y la Sección 7 la localiza numéricamente.

Lo que sí puede empeorar con la dimensión es la *constante* σ_f , no la tasa. La Sección 7 muestra un caso donde σ_f se vuelve catastrófica.

5.4. Intervalo de confianza del 95 %

El Teorema del Límite Central establece que, si $\sigma_f^2 < \infty$,

$$\frac{\hat{I}_N - I}{\text{EE}(\hat{I}_N)} \xrightarrow{d} \mathcal{N}(0, 1) \quad \text{cuando } N \rightarrow \infty.$$

Escribiendo $z_{0.975} \approx 1.96$ para el cuantil que deja 2.5 % en cada cola, para N grande

$$\mathbb{P} \left(-z_{0.975} \leq \frac{\hat{I}_N - I}{\text{EE}} \leq z_{0.975} \right) \approx 0.95.$$

Despejando I dentro de la desigualdad se obtiene el intervalo

$$\text{IC}_{95\%} = \left[\hat{I}_N - 1.96 \frac{(b-a)S_f}{\sqrt{N}}, \hat{I}_N + 1.96 \frac{(b-a)S_f}{\sqrt{N}} \right] \quad (11)$$

donde σ_f , que es desconocida, se reemplaza por su estimador muestral

$$S_f^2 = \frac{1}{N-1} \sum_{i=1}^N (f(U_i) - \bar{f})^2, \quad \bar{f} = \frac{1}{N} \sum_{i=1}^N f(U_i). \quad (12)$$

El divisor $N-1$ (corrección de Bessel) hace que S_f^2 sea insesgado para σ_f^2 ; con divisor N el estimador subestima sistemáticamente la varianza y el intervalo saldría demasiado angosto.

Qué significa realmente el intervalo. El 95 % no es la probabilidad de que I esté en *este* intervalo: I es una constante, o está o no está. Es la probabilidad de que el *procedimiento* produzca un intervalo que atrape a I , sobre repeticiones del experimento. Esa lectura es verificable, y se verifica en la Sección 6: al partir una corrida larga en 1000 bloques disjuntos, 946 de los intervalos contienen el valor verdadero, un 94.6 % frente al 95 % nominal.

Dos supuestos que conviene no olvidar. El intervalo (11) es *asintótico*: descansa en el TLC, así que para N pequeño o integrandos muy asimétricos la cobertura real puede alejarse del 95 %. Y exige $\sigma_f^2 < \infty$; con integrandos de varianza infinita el TLC no aplica y el intervalo carece de sentido. Un caso límite aparece en la Sección 7, donde en $d = 20$ ningún punto cae dentro de la región, $S_f = 0$ y el intervalo colapsa a un punto.

6. Integrales en una dimensión

Se estiman las dos integrales del enunciado,

$$(a) \int_0^1 \sin(\pi x) dx = \frac{2}{\pi} \approx 0.63661977, \quad (b) \int_0^2 \frac{1}{\sqrt{2\pi}} e^{-x^2/2} dx = \Phi(2) - \Phi(0) \approx 0.47724987,$$

usando el estimador (9) y el intervalo (11). Cada integral se calcula con dos fuentes de uniformes: el MT19937 implementado en la Parte 1 y el LCG de Numerical Recipes.

Integral	Fuente	$\hat{I}_N$	EE	IC 95%	Valor real	Error	Contiene
(a)	MT19937 (propio)	0.637372	0.000973	[0.635466, 0.639279]	0.636620	+7.53e-04	Sí
(a)	LCG (propio)	0.637499	0.000974	[0.635590, 0.639407]	0.636620	+8.79e-04	Sí
(b)	MT19937 (propio)	0.477405	0.000727	[0.475980, 0.478831]	0.477250	+1.55e-04	Sí
(b)	LCG (propio)	0.476433	0.000727	[0.475008, 0.477858]	0.477250	-8.17e-04	Sí

Tabla 12: Estimaciones con $N = 100\,000$. Los cuatro intervalos contienen el valor verdadero.

Los cuatro intervalos atrapan el valor real. Los errores son del orden de 10^{-4} , consistentes con un error estándar de $\sim 10^{-3}$: las estimaciones caen a menos de un error estándar del valor exacto, que es exactamente lo que predice la teoría.

6.1. Estudio de convergencia

La Figura 5 grafica el error absoluto medio contra N en escala log-log, con N desde 10^1 hasta 10^6 . Para que la recta sea legible se promedia el error sobre 20 réplicas en bloques disjuntos: una sola corrida produce una nube demasiado ruidosa para ajustar una pendiente con sentido.

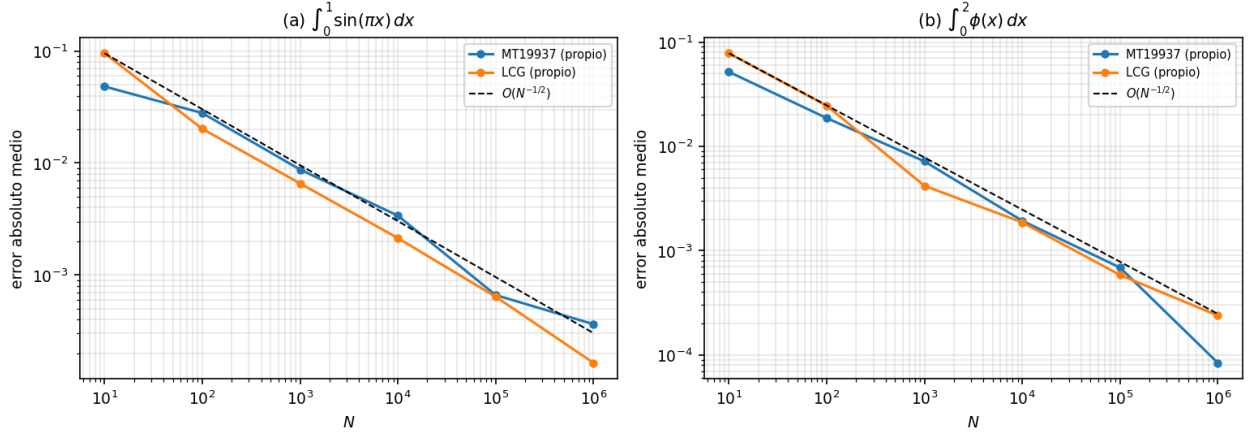


Figura 5: Error absoluto medio contra N . La recta discontinua tiene pendiente exacta $-1/2$.

Integral	Fuente	Pendiente ajustada	Teórica	Desvío
(a)	MT19937 (propio)	-0.4546	-0.5	0.0454
(a)	LCG (propio)	-0.5378	-0.5	0.0378
(b)	MT19937 (propio)	-0.5379	-0.5	0.0379
(b)	LCG (propio)	-0.5087	-0.5	0.0087

Tabla 13: Pendientes ajustadas por mínimos cuadrados sobre los logaritmos.

Si $|\hat{I}_N - I| \approx CN^{-1/2}$, entonces $\log |\hat{I}_N - I| \approx \log C - \frac{1}{2} \log N$, de modo que la pendiente en escala log-log debe ser $-1/2$. Las cuatro pendientes ajustadas quedan entre -0.45 y -0.54 , con desvíos de a lo sumo 0.046 respecto al valor teórico. **La tasa $O(N^{-1/2})$ derivada en el Teorema 5 queda confirmada empíricamente.**

Los desvíos no son un defecto del método sino ruido de estimación: el error de una corrida es una variable aleatoria, y promediar 20 réplicas reduce esa dispersión pero no la elimina. Con N grande el número de réplicas baja (para no agotar la memoria), lo que aumenta el ruido justo en el extremo derecho de la recta, que es donde más pesa el ajuste.

6.2. Verificación de la cobertura

El intervalo (11) promete atrapar el valor real el 95% de las veces. Eso es comprobable. Se genera una secuencia de 10^7 uniformes, se parte en 1000 bloques disjuntos de $N = 10^4$, se construye un intervalo por bloque y se cuenta cuántos contienen $2/\pi$:

$$946 \text{ de } 1000 = 94.6 \%, \quad \text{esperado } 950 \pm 6.9.$$

El resultado cae dentro de una desviación estándar del valor nominal. Se usan bloques disjuntos de una misma corrida, y no semillas distintas, porque en un LCG dos semillas producen secuencias relacionadas de forma determinista; los bloques disjuntos garantizan que ningún número se reutilice entre réplicas.

7. Alta dimensión y la maldición de la dimensionalidad

Se estima el volumen de la bola unitaria en d dimensiones, $V_d = \pi^{d/2}/\Gamma(d/2 + 1)$, muestreando puntos uniformes en el cubo $[-1, 1]^d$ y contando la fracción que cae dentro. El valor cerrado se usa únicamente como verdad de referencia.

Como la fracción p es una proporción binomial, $\text{Var}(\hat{p}) = p(1-p)/N$, y el intervalo se construye sobre esa varianza multiplicada por el volumen del cubo, 2^d .

d	Aciertos	Esperados	$\hat{V}_d$	IC 95%	V_d exacto	Error rel.	Contiene
2	157 087	157079.633	3.141740	[3.134543, 3.148937]	3.141593	0.00%	Sí
5	33 075	32898.681	5.292000	[5.239897, 5.344103]	5.263789	0.54%	Sí
10	491	498.079	2.513920	[2.291832, 2.736008]	2.550164	1.42%	Sí
20	0	0.005	0.000000	[0.000000, 15.728640]	0.025807	100.00%	Sí

Tabla 14: Estimación de V_d con $N = 200\,000$ puntos.

La columna de aciertos es la que cuenta la historia. En $d = 2$ caen dentro 157 087 puntos de 200 000; en $d = 10$ apenas 491; en $d = 20$, **ninguno**. Y no es un error: el número esperado de aciertos en $d = 20$ es 0.005, así que observar cero es el resultado más probable con diferencia.

El intervalo cuando no hay aciertos. Con $\hat{p} = 0$ el error estándar binomial $\sqrt{\hat{p}(1-\hat{p})/N}$ vale cero y el intervalo (11) colapsa a $[0, 0]$, que *no* contiene el valor real $V_{20} \approx 0.0258$. La aproximación normal exige varios aciertos para ser válida. En su lugar se usa la *regla de tres*: con cero éxitos en N ensayos, el límite superior del intervalo del 95 % para la proporción es aproximadamente $3/N$, lo que da $[0, 15.73]$. Contiene el valor real, pero es tan ancho que no informa nada. Esa inutilidad es el diagnóstico.

7.1. Comparación contra cuadratura determinística

d	Puntos	Evaluaciones	$\hat{V}_d$ rejilla	Error rel.
2	10^2	100	3.200000	1.86%
5	10^5	100 000	4.884480	7.21%
10	10^{10}	10 000 000 000	inviable	—
20	10^{20}	100 000 000 000 000 000 000	inviable	—

Tabla 15: Cuadratura en rejilla con $m = 10$ subdivisiones por eje.

Una rejilla de $m = 10$ por eje necesita 10^d evaluaciones. En $d = 2$ son 100 y en $d = 5$ son cien mil, ambas triviales. En $d = 10$ son **diez mil millones**, y en $d = 20$ son 10^{20} : más evaluaciones que segundos transcurridos desde el Big Bang. El costo crece exponencialmente con d mientras el refinamiento por eje se queda clavado en 10. *Eso* es la maldición de la dimensionalidad.

d	m	Presupuesto	Error rejilla	Error MC
2	447	199 809	0.00%	0.00%
5	11	161 051	2.35%	0.39%
10	3	59 049	36.68%	9.56%
20	2	1 048 576	100.00%	100.00%

Tabla 16: Error a igual presupuesto de evaluaciones: se elige m tal que $m^d \approx 200\,000$.

A igual presupuesto el cruce es nítido. En $d = 2$ empatan. En $d = 5$ Monte Carlo ya gana por un factor de seis. En $d = 10$ el presupuesto solo alcanza para $m = 3$ subdivisiones por eje, y la rejilla se equivoca por un 36.7 % contra el 9.6 % de Monte Carlo. La rejilla no falla por ser mala regla, sino porque en alta dimensión el presupuesto no da ni para tres puntos por eje.

7.2. Por qué el muestreo uniforme se vuelve ineficiente

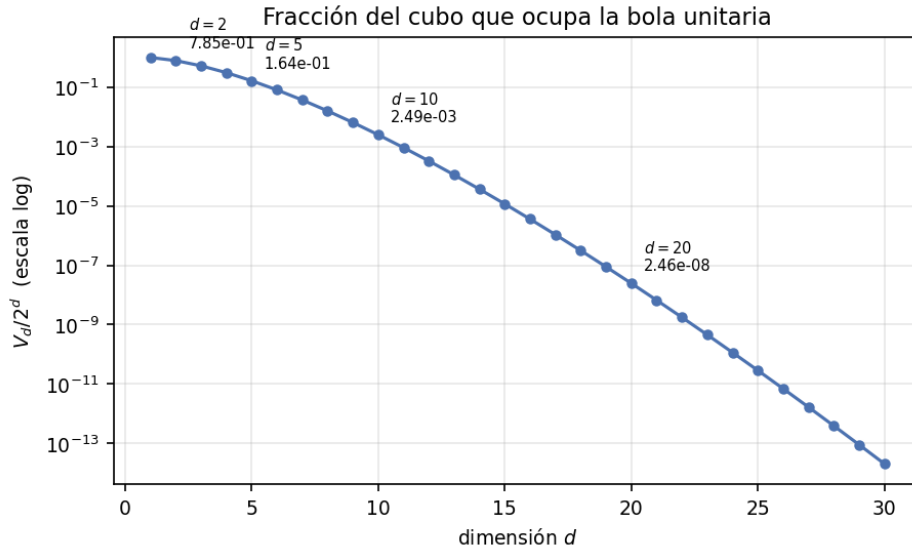


Figura 6: Fracción del cubo $[-1, 1]^d$ ocupada por la bola unitaria, en escala logarítmica.

La fracción $V_d/2^d$ se desploma: 0.785 en $d = 2$, 0.165 en $d = 5$, 2.5×10^{-3} en $d = 10$, 2.5×10^{-8} en $d = 20$ y 2.0×10^{-14} en $d = 30$.

El fenómeno geométrico es que **en alta dimensión, casi todo el volumen de un cubo está en sus esquinas**. La bola inscrita toca cada cara a distancia 1 del centro, pero los vértices del cubo están a distancia $\sqrt{d}$, que crece sin límite. El cubo se vuelve, en esencia, 2^d púas delgadas alrededor de una bola comparativamente diminuta.

Esto no contradice la tasa $O(N^{-1/2})$: la tasa sigue siendo $N^{-1/2}$, pero la *constante* explota. Con p minúsculo, el error relativo del estimador de proporción es

$$\frac{EE(\hat{p})}{p} = \sqrt{\frac{1-p}{Np}} \approx \frac{1}{\sqrt{Np}},$$

así que mantener un error relativo fijo exige $N \propto 1/p = 2^d/V_d$, que en $d = 20$ significa del orden de 10^8 puntos para un solo dígito de precisión.

La lección de diseño es que el muestreo uniforme en el cubo es la elección equivocada: gasta prácticamente todas sus muestras en una región que no aporta. Un buen estimador en alta dimensión debe concentrar el muestreo donde el integrando no se anula, que es la idea del muestreo por importancia.

8. Reducción de varianza

Como el error es $\sigma/\sqrt{N}$, bajarlo admite dos caminos: subir N , que cuesta tiempo lineal, o bajar σ . Se implementan dos técnicas y se aplican a *ambas* integrales de la Sección 6, porque el contraste entre ellas es el resultado más interesante.

8.1. Variables antitéticas

A cada U se le empareja su reflejo $a+b-U$, que tiene la misma distribución. Con $N/2$ uniformes se producen N evaluaciones, de modo que el presupuesto se mantiene. Escribiendo $Y = f(U)$ e $Y' = f(a+b-U)$,

$$\text{Var}\left(\frac{Y+Y'}{2}\right) = \frac{\text{Var}(Y) + \text{Var}(Y') + 2\text{Cov}(Y, Y')}{4} = \frac{\text{Var}(Y)(1+\rho)}{2}, \quad (13)$$

donde ρ es la correlación entre Y e Y' . La técnica ayuda si y solo si $\rho < 0$, lo que ocurre cuando f es monótona: si U es grande, $a+b-U$ es pequeño, y una f monótona convierte eso en valores opuestos que se compensan.

8.2. Variables de control

Se usa una variable $g(X)$ de media conocida μ_g y se corrige: $Y_c = Y - c(g(X) - \mu_g)$. Como $\mathbb{E}[g(X) - \mu_g] = 0$, el estimador sigue siendo insesgado para cualquier c . Su varianza es

$$\text{Var}(Y_c) = \text{Var}(Y) - 2c\text{Cov}(Y, g) + c^2\text{Var}(g),$$

una parábola en c cuyo mínimo está en $c^* = \text{Cov}(Y, g)/\text{Var}(g)$. Sustituyendo,

$$\text{Var}(Y_{c^*}) = \text{Var}(Y)(1 - \rho^2) \implies \text{factor de reducción} = \frac{1}{1 - \rho^2}, \quad (14)$$

con ρ la correlación entre f y g . La técnica vale exactamente lo que g se parezca a f .

8.3. Resultados

Integral	Técnica	$\hat{I}$	Varianza	Factor	Speedup	IC 95%
(a)	Simple	0.637478	9.4568e-02	1.00	1.00	[0.636130, 0.638825]
(a)	Antitéticas	0.637372	9.4585e-02	0.50	0.50	[0.635466, 0.639279]
(a)	Control $g(x) = x$	0.637482	9.4566e-02	1.00	1.00	[0.636135, 0.638830]
(a)	Control $g(x) = x(1-x)$	0.636617	2.9781e-04	317.54	317.54	[0.636542, 0.636693]
(b)	Simple	0.478216	5.2886e-02	1.00	1.00	[0.477208, 0.479224]
(b)	Antitéticas	0.477270	7.2851e-05	362.97	362.97	[0.477217, 0.477323]
(b)	Control $g(x) = x$	0.477273	4.9911e-04	105.96	105.96	[0.477175, 0.477371]
(b)	Control $g(x) = x(1-x)$	0.477190	9.9949e-03	5.29	5.29	[0.476752, 0.477628]

Tabla 17: Factores de reducción con $N = 200\,000$ evaluaciones en todos los casos.

¿La técnica siempre ayuda? No, y las dos integrales lo demuestran en direcciones opuestas.

Antitéticas. En la integral (b), donde ϕ es monótona decreciente en $[0, 2]$, el factor es **363**×. En la integral (a) el factor es **0.50**×: la técnica **empeora** el estimador. La razón es exacta y no estadística: $\sin(\pi(1-x)) = \sin(\pi x)$, de modo que $Y' = Y$ idénticamente, $\rho = +1$, y (13) da $\text{Var}(Y)$, la misma varianza que Monte Carlo simple, pero habiendo gastado dos evaluaciones por muestra en lugar de una. La simetría del integrando respecto al centro del intervalo es precisamente el peor caso.

Variables de control. En la integral (a), el control lineal $g(x) = x$ da $\rho = 0.004$ y un factor de **1.00**×: no hace nada. Otra vez la simetría es la culpable, ahora porque $\text{Cov}(\sin(\pi U), U) = 0$ para $U \sim \text{Unif}(0, 1)$. Basta cambiar a un control que comparta la simetría, $g(x) = x(1-x)$, para obtener $\rho = 0.9984$ y un factor de **317**×. En la integral (b) el orden se invierte: el control lineal da **106**× y el cuadrático solo **5.3**×.

Contraste con la predicción teórica. La fórmula (14) predice un factor $1/(1-\rho^2)$. Con $\rho = 0.9984$ predice 313 y se midió **317**; con $\rho = -0.9953$ predice 107 y se midió **106**. La coincidencia confirma la derivación.

Speedup efectivo. Como la varianza del promedio es inversamente proporcional a N , reducir la varianza por un factor k equivale a multiplicar el número de muestras por k . El control $g(x) = x(1-x)$ en la integral (a) rinde como $317 \times 200\,000 \approx 6.3 \times 10^7$ muestras de Monte Carlo simple, a costo de una sola evaluación extra por punto.

9. Cuando el generador arruina el resultado: RANDU

Se estima la integral triple

$$I = \iiint_{[0,1]^3} \mathbf{1}[x^2 + y^2 + z^2 \leq 1] \, dx \, dy \, dz = \frac{\pi}{6} \approx 0.52359878,$$

que depende genuinamente de ternas (x, y, z) , usando ternas consecutivas y disjuntas de cada generador.

Fuente	$\hat{I}$	IC 95%	Error	error /EE	Contiene $\pi/6$
RANDU	0.523535	[0.522843, 0.524227]	-6.38e-05	0.2	Sí
Mersenne Twister	0.523711	[0.523019, 0.524404]	+1.13e-04	0.3	Sí
LCG (Num. Recipes)	0.522999	[0.522307, 0.523691]	-6.00e-04	1.7	Sí

Tabla 18: Integral triple con 2 000 000 de ternas.

Un resultado contrario a la expectativa. RANDU **no falla**. Su error es de 0.2 errores estándar, mejor incluso que el del LCG de Numerical Recipes (1.7). Los tres intervalos contienen $\pi/6$.

Esto merece explicación en vez de disimulo. Los 15 planos derivados en el Teorema 3 tienen normal $\mathbf{h} = (9, -6, 1)$ y están separados por

$$\frac{1}{\|\mathbf{h}\|} = \frac{1}{\sqrt{81 + 36 + 1}} = \frac{1}{\sqrt{118}} \approx 0.0921.$$

La bola unitaria restringida a $[0, 1]^3$ mide del orden de 1 en cada dirección, así que la atraviesan una docena de planos. Repartir los puntos sobre planos equiespaciados que cortan la región muchas veces se parece mucho a un muestreo *estratificado*, y la estratificación no sesga: si acaso, reduce varianza. El defecto de RANDU no desaparece, simplemente no muerde a esta escala.

9.1. Dónde sí muerde

Si la estructura tiene una escala característica de 0.0921, el fallo debe aparecer al estimar regiones de ese tamaño o menores. Se repite el experimento con bolas centradas en $(\frac{1}{2}, \frac{1}{2}, \frac{1}{2})$ de radio decreciente.

r	V real	RANDU	Error	Mersenne Twister	Error
0.30	1.131e-01	1.140e-01	0.8%	1.132e-01	0.1%
0.15	1.414e-02	1.480e-02	4.7%	1.422e-02	0.6%
0.08	2.145e-03	1.859e-03	13.3%	2.238e-03	4.4%
0.05	5.236e-04	7.330e-04	40.0%	5.550e-04	6.0%
0.03	1.131e-04	2.750e-04	143.2%	1.260e-04	11.4%

Tabla 19: Volumen de bolas pequeñas, 10^6 ternas. El espaciado entre planos de RANDU es 0.0921.

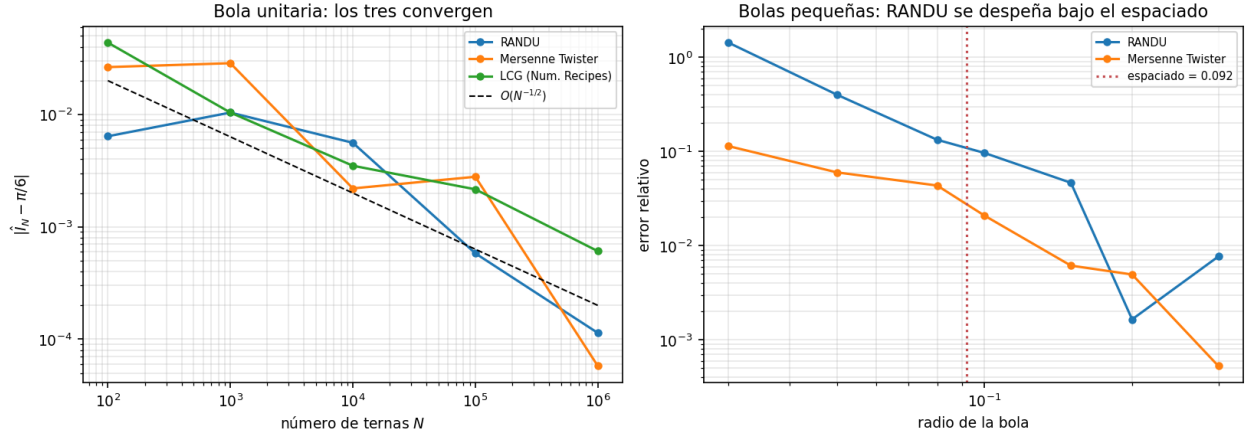


Figura 7: Izquierda: los tres generadores convergen para la bola unitaria. Derecha: al bajar el radio por debajo del espaciado entre planos (línea punteada), RANDU se despeña mientras Mersenne solo acusa el ruido estadístico de tener menos aciertos.

El error de RANDU pasa de 0.8 % con radio 0.30 a **143 %** con radio 0.03, mientras Mersenne Twister solo se degrada de 0.1 % a 11.4 % por el ruido esperable de tener pocos aciertos. El cruce ocurre alrededor del radio 0.05, es decir cuando el diámetro de la bola es comparable al espaciado entre planos.

La explicación geométrica. Una bola de radio 0.03 tiene diámetro 0.06, menor que la separación 0.0921 entre planos consecutivos. Según dónde caiga, o bien ningún plano la atraviesa —y RANDU no puede colocar un solo punto dentro, estimando volumen cero— o bien un plano la corta por el medio y todos los puntos de RANDU en esa zona se concentran en un disco, sobrestimando. No hay término medio, porque **RANDU no tiene puntos entre los planos**. El 143 % de error de la tabla es este segundo caso.

Conexión con la prueba espectral. Esto es la Sección 3 vista desde Monte Carlo. Allí se comprobó que $k_i = 9u_i - 6u_{i+1} + u_{i+2}$ toma exactamente 15 valores enteros, con desviación 0.00 respecto al entero más cercano en las 19998 ternas examinadas. Aquí se mide el precio de esa estructura: el estimador falla cuando la región de interés es más fina que la retícula. La prueba espectral *predice* la escala a la que el estimador va a fallar, y la predicción se cumple.

La moraleja, con un matiz. Un método de Monte Carlo nunca será mejor que su fuente de aleatoriedad. Pero el experimento obliga a precisar la frase: la calidad de un generador no es un número único, sino que depende de la escala a la que se lo interroga. RANDU es perfectamente utilizable para estimar el volumen de la bola unitaria y catastrófico para una bola de radio 0.03, con los mismos puntos y en el mismo cubo. Lo peligroso no es que un generador sea malo: es que sea malo *solo* en el régimen que uno necesita, mientras pasa todas las pruebas que uno corre.

Referencias

- [1] T. E. Hull y A. R. Dobell. “Random Number Generators”. *SIAM Review* 4.3 (1962), pp. 230–254. doi: 10.1137/1004061.
- [2] G. Marsaglia. “Random Numbers Fall Mainly in the Planes”. *Proceedings of the National Academy of Sciences* 61.1 (1968), pp. 25–28. doi: 10.1073/pnas.61.1.25.
- [3] D. E. Knuth. *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*. 3.^a ed. Addison-Wesley, 1997. Capítulo 3.
- [4] J. von Neumann. “Various Techniques Used in Connection with Random Digits”. *National Bureau of Standards Applied Mathematics Series* 12 (1951), pp. 36–38.
- [5] M. Matsumoto y T. Nishimura. “Mersenne Twister: A 623-Dimensionally Equidistributed Uniform Pseudo-Random Number Generator”. *ACM Transactions on Modeling and Computer Simulation* 8.1 (1998), pp. 3–30. doi: 10.1145/272991.272995.
- [6] L. Blum, M. Blum y M. Shub. “A Simple Unpredictable Pseudo-Random Number Generator”. *SIAM Journal on Computing* 15.2 (1986), pp. 364–383. doi: 10.1137/0215025.
- [7] A. M. Law. *Simulation Modeling and Analysis*. 5.^a ed. McGraw-Hill, 2015.
- [8] RANDOM.ORG. *Introduction to Randomness and Random Numbers*. <https://www.random.org/randomness/>.